

ISIS Capitalist

Partie backend



nicolas.singer@gmail.com

La partie backend du projet va se réaliser en node.js. Nous faisons ici le choix d'utiliser le framework NestJS (<https://docs.nestjs.com/>) pour les facilités qu'il apporte tant pour l'amorçage du projet que pour son intégration de différentes librairies qui nous seront utiles.

Organisation des fichiers du projet

On vous recommande de gérer les parties front et back du projet dans deux dossiers différents, contenus dans un dossier unique qui sera placé sous git.

```
isiscapitalist/  
- frontend/  
- backend/
```

Dans cet énoncé nous créons le dossier `backend` et son contenu.

Amorçage du projet

Placez-vous dans le dossier racine du projet et créez le dossier `backend` à partir d'un squelette préfabriqué avec les commandes ci-dessous saisie dans un terminal :

```
git clone https://github.com/nestjs/typescript-starter.git backend  
cd backend  
npm install
```

Vous pouvez ensuite ouvrir le dossier `backend` dans votre éditeur favori.

Ajout du schéma GraphQL et des classes métiers

Dans un but d'interopérabilité, le schéma GraphQL vous est fourni. Nous allons implémenter le service GraphQL à partir de lui. Cette approche porte le nom de **schema first** par opposition à l'approche **code first** qui consisterait à partir du code pour obtenir le schéma.

Dans un terminal, ajoutez les dépendances nécessaires à l'utilisation de GraphQL avec les commandes suivantes :

```
npm i @nestjs/graphql @nestjs/apollo @apollo/server graphql ts-morph
```

Dans le dossier `src` du projet, créez un fichier appelé `schema.graphql` et dotez-le du schéma suivant :

```
enum RatioType {
  gain
  vitesse
  ange
}

type Palier {
  name: String!
  logo: String!
  seuil: Float!
  idcible: Int!
  ratio: Int!
  typeratio: RatioType!
  unlocked: Boolean!
}

type Product {
  id: Int!
  name: String!
  logo: String!
  cout: Float!
  croissance: Float!
  revenu: Float!
  vitesse: Int!
  quantite: Int!
  timeleft: Int!
  managerUnlocked: Boolean!
  paliers: [Palier!]!
}

type world {
  name: String!
  logo: String!
  money: Float!
  score: Float!
  totalangels: Int!
  activeangels: Int!
  angelbonus: Int!
  lastupdate: Int!
  products: [Product!]!
  allunlocks: [Palier!]!
  upgrades: [Palier!]!
  angelupgrades: [Palier!]!
  managers: [Palier!]!
}

type Query {
  getworld(user: String!): world
}
```

```

}

type Mutation {
  acheterQtProduit(user: String!, id: Int!, quantite: Int!): Product
  lancerProductionProduit(user: String!, id: Int!): Product
  engagerManager(user: String!, name: String!): Palier
  acheterCashUpgrade(user: String!, name: String!): Palier
  acheterAngelUpgrade(user: String!, name: String!): Palier
  resetWorld(user: String!): World
}

```

Nous pourrions créer manuellement les classes *métiers* Typescript définies par ce schéma, mais la bibliothèque GraphQL a des outils qui nous permettent de rendre cette génération automatique. L'activation de ces outils se fait en ajoutant dans la partie `import` du fichier `app.module.ts` le code suivant :

```

@Module({
  imports: [
    GraphQLModule.forRoot<ApolloDriverConfig>({
      driver: ApolloDriver,
      typePaths: ['./**/*.graphql'],
      definitions: {
        path: join(process.cwd(), 'src/graphql.ts'),
        outputAs: 'class',
      },
    }),
  ],
  controllers: [AppController],
  providers: [AppService],
})

```

La génération des classes Typescript et l'activation du *playground* se fait en démarrant l'application en mode développement avec la commande :

```
npm run start:dev
```

Suite au démarrage, vérifiez qu'un fichier `graphql.ts` est bien apparu dans le dossier `src` et qu'il contient la définition des classes Typescript représentant les différents objets du monde, en particulier les classes `World`, `Product`, et `Palier`. **Ces classes seront aussi utilisées par la partie frontend.**

Vérifiez aussi que le *playground* s'est bien lancé en connectant votre navigateur sur l'URL <http://localhost:3000>.

Création du monde

Maintenant que tout est en place, créez une représentation de votre monde au format JSON dans un nouveau fichier que vous appellerez `origworld.ts` placé dans le dossier `src`.

Le squelette de ce fichier est donné ci-dessous pour deux produits, chaque produit disposant pour l'exemple de deux paliers d'*unlock*. Figurent aussi deux *allunlocks*, deux *upgrades*, deux *angleupgrades* et les deux managers des produits.

Editez ce fichier pour construire votre propre monde qui devra contenir 6 produits et donc 6 manageurs, au moins trois *unlocks* par produits, au moins trois *allunlocks*, une dizaine d'upgrades et quelques *angelupgrades*.

Certains éléments possèdent des images, voir plus loin pour savoir où les placer.

```
import { RatioType } from './graphql';

export const origworld = {
  name: 'A Nice World 2',
  logo: 'icones/ecolo.jpg',
  money: 0,
  score: 0,
  totalangels: 0,
  activeangels: 0,
  angelbonus: 2,
  lastupdate: 0,
  products: [
    {
      id: 1,
      name: 'Paper Bags',
      logo: 'icones/sacpapier.jpg',
      cout: 4,
      croissance: 1.07,
      revenu: 1,
      vitesse: 500,
      quantite: 1,
      timeleft: 0,
      managerUnlocked: false,
      paliers: [
        {
          name: 'Paper is beautiful !',
          logo: 'icones/sacpapier.jpg',
          seuil: 20,
          idcible: 1,
          ratio: 2,
          typeratio: RatioType.vitesse,
          unlocked: false,
        },
        {
          name: "Don't forget your paper bag !",
          logo: 'icones/sacpapier.jpg',
          seuil: 75,
          idcible: 1,
          ratio: 2,
          typeratio: RatioType.vitesse,
          unlocked: false,
        },
      ],
    },
    {
      id: 2,
      name: 'Recycle Bins',
      logo: 'icones/recyclage.jpg',
      cout: 60,
```

```
croissance: 1.15,
revenu: 60,
vitesse: 3000,
quantite: 0,
timeleft: 0,
managerUnlocked: false,
paliers: [
  {
    name: 'Give me some good bins !',
    logo: 'icones/recyclage.jpg',
    seuil: 20,
    idcible: 2,
    ratio: 2,
    typeratio: RatioType.vitesse,
    unlocked: false,
  },
  {
    name: 'My Trashcans are better than yours ! ',
    logo: 'icones/recyclage.jpg',
    seuil: 75,
    idcible: 2,
    ratio: 2,
    typeratio: RatioType.vitesse,
    unlocked: false,
  }
],
allunlocks: [
  {
    name: 'All is better than one',
    logo: 'icones/all.jpg',
    seuil: 30,
    idcible: 0,
    ratio: 2,
    typeratio: RatioType.gain,
    unlocked: false,
  },
  {
    name: 'To take and not to give',
    logo: 'icones/all.jpg',
    seuil: 150,
    idcible: 0,
    ratio: 3,
    typeratio: RatioType.gain,
    unlocked: false,
  }
],
upgrades: [
  {
    name: 'Do you like paper bag ?',
    logo: 'icones/sacpapier.jpg',
    seuil: 1000,
    idcible: 1,
    ratio: 3,
    typeratio: RatioType.gain,
    unlocked: false,
```

```

    },
    {
      name: 'This is my bin',
      logo: 'icones/recyclage.jpg',
      seuil: 15000,
      idcible: 2,
      ratio: 3,
      typeratio: RatioType.gain,
      unlocked: false,
    },
  ],
  angelupgrades: [
    {
      name: 'Angel Sacrifice',
      logo: 'icones/angel.png',
      seuil: 10,
      idcible: 0,
      ratio: 3,
      typeratio: RatioType.gain,
      unlocked: false,
    },
    {
      name: 'Angelic Mutiny',
      logo: 'icones/angel.png',
      seuil: 100000,
      idcible: -1,
      ratio: 2,
      typeratio: RatioType.ange,
      unlocked: false,
    },
  ],
  managers: [
    {
      name: 'Wangari Maathai',
      logo: 'icones/WangariMaathai.jpg',
      seuil: 10,
      idcible: 1,
      ratio: 0,
      typeratio: RatioType.gain,
      unlocked: false,
    },
    {
      name: 'Ellen MacArthur',
      logo: 'icones/ellenmacarthur.jpg',
      seuil: 15000,
      idcible: 2,
      ratio: 0,
      typeratio: RatioType.gain,
      unlocked: false,
    },
  ],
};

```

Stockez vos images dans un dossier `icones` que vous placerez dans un dossier `public` du dossier `backend`, comme illustré sur l'arborescence ci-dessous :

```
backend/  
- public/  
  - icones/  
    - image1.jpg  
    - image2.jpg
```

Pour que le backend puisse servir ces fichiers statiques, vous devez modifier le fichier `main.ts` pour qu'il devienne :

```
async function bootstrap() {  
  const app = await NestFactory.create<NestExpressApplication>(AppModule);  
  app.useStaticAssets(join(__dirname, '..', 'public'));  
  app.enableCors();  
  await app.listen(process.env.PORT ?? 3000);  
}  
bootstrap();
```

Vérifiez que vos images sont accessibles en dirigeant votre navigateur vers l'adresse <http://localhost:3000/icones/images1.jpg>

Création du *resolver*

Il faut maintenant implémenter les *query* et les *mutation* du schéma. C'est le rôle d'un composant appelé le *resolver*. Créez dans le dossier `src` un nouveau fichier appelé `resolver.ts`. Dotez ce fichier du squelette suivant :

```
import { Origworld } from './origworld';  
import { Args, Mutation, Query, Resolver } from '@nestjs/graphql';  
import { AppService } from './app.service';  
import { Palier } from './graphql';  
  
@Resolver('World')  
export class GraphQLResolver {  
  constructor(private service: AppService) {}  
  
  @Query()  
  async getWorld(@Args('user') user: string) {  
    const world = this.service.readUserWorld(user);  
    return world;  
  }  
}
```

Déclarez le *resolver* dans le projet en ajoutant sa classe dans les providers du fichier `app.module.ts` comme ceci :

```
providers: [AppService, GraphQLResolver],
```

Le *resolver* ci-dessous implémente la *query* `getworld` définie par le schéma. Son rôle est d'aller chercher le monde qui correspond à l'utilisateur passé en paramètre et de le retourner. On voit en effet qu'on récupère le paramètre `user` de la *query* et qu'on appelle la méthode `readuserworld` du service `AppService`, méthode qu'il va nous falloir définir. L'idée est en effet de placer dans le composant service la partie commune du code de l'application, et de mettre dans chacun des *resolvers*, le code qui lui est propre.

Passez à présent du côté du service dans le fichier `app.service.ts`. La méthode `readuserworld(user: string)` que nous devons définir, doit charger le monde de l'utilisateur passé en paramètre, le retourner s'il existe, et sinon retourner le monde d'origine.

Pour stocker les mondes, nous allons faire l'économie d'une base de données, et les stocker directement sous forme de fichiers dans un dossier que vous devez créer dans le dossier `backend`, et que vous appellerez `userworlds`. Chaque monde sera nommé avec le nom de son utilisateur suivi de `-world.json`.

La méthode de lecture du monde, doit donc ouvrir le fichier `{user}-world.json`, le retourner s'il existe, et sinon retourner la contenu de la variable `origworld` définie dans le fichier `origworld.ts`.

Voici la méthode à ajouter dans le fichier `app.service.ts` qui réalise cela :

```
import * as fs from 'fs';
import * as path from 'path';

@Injectable()
export class AppService {
  readuserworld(user: string): world {
    try {
      const data = fs.readFileSync(
        path.join(process.cwd(), 'userworlds/', user + '-world.json'),
      );
      return JSON.parse(data.toString());
    } catch (e: unknown) {
      console.log((e as Error).message);
      return origworld;
    }
  }
}
```

On voit que la méthode tente d'ouvrir le monde de l'utilisateur, et que si cela génère une exception, on retourne le monde d'origine.

Voilà votre *query* implémentée, testez là dans le *playground* et vérifiez qu'elle fonctionne et retourne bien le monde d'origine quelque soit l'utilisateur (puisque pour l'instant nous n'avons pas encore sauvegardé nos mondes et qu'il n'existe pas encore de monde propre à un utilisateur)

Plus tard, vous allez implémenter des mutations qui vont modifier le monde. Vous aurez donc besoin d'une méthode qui écrase le monde de l'utilisateur par sa nouvelle version.

Voici cette méthode, à ajouter dans le fichier `app.service.ts` :


```

saveWorld(user: string, world: World) {
  fs.writeFile(
    path.join(process.cwd(), 'userworlds/', user + '-world.json'),
    JSON.stringify(world),
    (err) => {
      if (err) {
        console.error(err);
        throw new Error(`Erreur d'écriture du monde côté serveur`);
      }
    },
  );
}

```

Elle prend en paramètre le monde à sauvegarder, et l'utilisateur pour lequel il faut l'écrire.

Vous devrez appeler cette méthode à la fin de chacune de vos *query* et *mutation* pour sauvegarder l'état du monde après changement, y compris à la fin du *resolver* `getWorld`. Faites-le et vérifiez en exécutant la *query* `getWorld` dans le *playground* que cela a pour effet de créer un nouveau fichier dans le dossier `userworlds`.

Implémentation des mutations

Il faut à présent implémenter le *resolver* de chacune des mutations.

Commençons par `acheterQtProduit`. Voici le squelette qui met en place la mutation dans le *resolver* :

```

@Mutation()
async acheterQtProduit(
  @Args('user') user: string,
  @Args('id') id: number,
  @Args('quantite') quantite: number,
) {
  // à compléter
}

```

La méthode à compléter doit :

- obtenir le monde du joueur en appelant la méthode `readUserWorld` du service
- aller chercher dans ce monde, le produit d'identifiant passé en paramètre, sachant que s'il n'existe pas, il convient de générer une erreur sous la forme d'une exception comme ceci :

```

throw new Error(`Le produit avec l'id ${args.id} n'existe pas`)

```

- augmenter sa quantité en ajoutant à la quantité actuellement possédée, la valeur du paramètre `quantite`. Mais cela ayant un coût, il faut déduire de l'argent du monde, le coût de l'achat. Ensuite il faut mettre à jour le coût d'achat du produit, puisque ce coût doit toujours représenter le coût d'achat du prochain produit.
- Sauvegarder le monde pour mémoriser les changements opérés en appelant la méthode `saveWorld` du service.

Avec le *playground*, vérifiez le bon fonctionnement de la mutation. Vous devez voir que le monde évolue, avec de l'argent en moins, et une quantité du produit qui augmente.

Passons à présent à `lancerProductionProduit`. Il faut comme précédemment aller chercher le produit du monde possédant l'identifiant passé en paramètre, puis affecter sa propriété `vitesse` à sa propriété `timeleft`. Autrement dit quand le serveur apprend que le client a lancé la production d'un produit, il se contente de noter le temps qu'il reste au produit pour terminer sa production. On verra plus tard comment le serveur sait que la production est terminée.

Implémentez également la mutation `engagerManager`. Sa méthode doit aller chercher dans le monde le manager de nom passé en paramètre, repérer de quel produit il est un manager, et aller chercher ce produit. Ensuite, on se borne à débloquent le manager en passant à vrai la propriété `managerUnlocked` du produit, ainsi que la propriété `unlocked` du manager.

Pour vérifier les deux dernières mutations, il nous manque un service essentiel, celui qui calcule l'évolution du monde en fonction du temps qui passe. En effet, entre deux actions du joueur, il faut calculer tous les éventuels gains que le joueur a pu faire entre temps.

Il vous reste donc à ajouter au service, une méthode qui calcule l'évolution de l'argent du joueur en fonction du temps qui s'est écoulé depuis la dernière mise à jour repéré par la propriété `lastupdate` du monde. Pour l'essentiel cette méthode doit parcourir chaque produit et pour chacun calculer combien d'exemplaires de ce produit ont été créés depuis la dernière mise à jour.

Si ce produit n'a pas de manager, il suffit de vérifier que son `timeleft` n'est pas nul, et qu'il est inférieur au temps écoulé. Si c'est le cas, un produit a été créé (et donc on ajoute les gains au score et à l'argent), sinon on met à jour `timeleft` en soustrayant le temps écoulé.

Si ce produit a un manager, c'est plus compliqué car il faut calculer combien de fois sa production complète a pu se produire depuis la dernière mise à jour, et mettre à jour le `timeleft` du produit en conséquence.

Le code final de cette méthode n'est pas très long mais vous demandera sans doute un peu de réflexion pour obtenir quelque chose qui fonctionne proprement. N'oubliez pas à chaque mis à jour de repositionner le `lastupdate` du monde sur l'instant courant.

Un échantillon de test a été placé à votre disposition à l'adresse <https://gitlab.com/-/snippets/2522185> pour que vous puissiez vérifier le bon fonctionnement de cette méthode.

Les unlocks

Quand vous avez conçu la méthode `acheterQtProduit` vous n'avez peut-être pas vérifié le déblocage éventuels des paliers d'*unlock*. Les *unlocks* des produits se débloquent quand la quantité d'un produit dépasse un certain seuil indiqué par l'objet `Palier`. Il faut donc vérifier cela pour le produit à chaque achat. Il faut aussi vérifier les paliers de *allunlocks* qui se débloquent quand tous la quantité de chaque produit est supérieure à un certain seuil. Ajoutez le code réalisant ces vérifications, débloquent les bons paliers, et appliquant leur bonus. Ce code doit être placé dans le service et appelé par le résolveur `acheterQtProduit`.

Les upgrades

Implémentez à présent les mutations `acheterCashUpgrade` et `acheterAngelUpgrade` qui permettent de débloquent des bonus avec respectivement de l'argent ou des anges. En principe vous devriez réutiliser une partie du code lié au déblocage des *unlock*.

Le reset du monde

Implémentez la mutation permettant la remise à zéro du monde. Cette action doit essentiellement avoir pour effet de :

- calculer le nombre d'anges supplémentaires gagnés lors de la partie en cours.
- remettre le monde dans son état initial en conservant néanmoins son score, et avec les deux propriétés relatives aux anges correctement positionnées.

Pour le premier point on ajoutera simplement les anges gagnés aux propriétés du monde `totalangels` et `activeangels`.

Pour le deuxième point, le plus simple est de repartir du monde original (celui servi à un nouveau joueur) en initialisant ses propriétés `score`, `totalangels` et `activeangels` aux bonnes valeurs.

Enfin il reste à modifier les fonctions de calcul du score pour qu'elles appliquent le bonus de revenu par ange actif. Ce bonus pouvant évoluer, on le trouvera dans la propriété `angelbonus` du monde (initialement il est fixé à 2%).

Le revenu gagné par la production d'un produit est alors de

$product.quantite * product.revenu * (1 + world.activeangels * world.angelbonus / 100)$